

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Actividad 2: Reporte de lectura (Java Syntax)

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

id634296@gmail.com

7CM2

31 de agosto de 2026

Índice

1. Introducción	2
2. Estructura de un archivo Java	2
3. Identificadores y tipos de variable	2
4. Encapsulamiento: mutators y accessors	2
5. Constructores	3
6. Herencia, clases abstractas e interfaces	3
7. Enums	3
8. Excepciones: checked y unchecked	4
9. Generics	4
10. Conclusión	4
11. Referencias Bibliográficas	5

Índice de figuras

1. Jerarquía de `Throwable` y ubicación de las excepciones checked y unchecked 4

Índice de tablas

1. Diferencias entre clase abstracta e interfaz 3

1. Introducción

Este reporte cubre la sintaxis base de Java: la estructura de un archivo fuente, el encapsulamiento, los constructores, la herencia con clases abstractas e interfaces, los enums, las excepciones y los generics.

Está organizado alrededor de los conceptos que sostienen el resto de la sintaxis (el diseño de clases, el manejo de errores y la reutilización de tipos) y los ilustra con un ejemplo propio: una clase `Carrera` y su correspondiente `CarreraDAO` para el acceso a datos.

2. Estructura de un archivo Java

Todo archivo fuente de Java empieza con la declaración de `package`, que agrupa las clases en un espacio de nombres y corresponde a la ruta de carpetas donde viven. Después va la sección de `import`, con los nombres completos de las clases, interfaces y enums que el archivo usa. Hay una distinción entre el `import` normal y el `import` estático: este último permite usar variables y métodos estáticos de otra clase sin escribir el nombre de esa clase cada vez, como en `import static java.lang.Math.PI`, que deja escribir `PI` en lugar de `Math.PI`. Cuando se usan muchas clases del mismo paquete, o muchos miembros estáticos de la misma clase, el `import` se puede compactar con un asterisco.

El resto de las reglas son básicas pero exigentes: Java distingue mayúsculas de minúsculas, así que `miVariable` y `mivariable` son dos identificadores distintos; todo excepto `import`, `package` y las declaraciones de clase, interfaz o enum va encerrado entre llaves; y cada instrucción termina en punto y coma. Son reglas fáciles de pasar por alto al empezar, y el error más común es justamente confundir los dos delimitadores: cerrar un constructor vacío con punto y coma en vez de llevarlo entre llaves, que es una declaración incompleta y no compila.

3. Identificadores y tipos de variable

Un identificador no puede ser una palabra reservada ni un literal booleano, y por convención se escribe en camelCase: minúscula inicial para atributos y métodos, mayúscula inicial para nombres de clase. Hay tres tipos de variable según dónde se declaran: los *fields* viven en el cuerpo de la clase, fuera de cualquier método; las *local variables* viven dentro de un método y solo existen ahí; y las *static variables* llevan el modificador `static` y pertenecen a la clase completa, no a cada instancia por separado.

Esta última categoría es la que sostiene el patrón que se usó para declarar el SQL en `CarreraDAO`:

Código 1: Campo estático, final y privado usado como constante

```
1 private static final String SQL_INSERT =  
2     "INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)";
```

`static` hace que exista una sola copia del texto para toda la clase, no una por cada objeto `CarreraDAO` que se cree. `final` impide que se reasigne después de la primera vez. Java no tiene la palabra reservada `const`; esta combinación es lo más cercano que el lenguaje ofrece a una constante, y de ahí sale también la convención de escribirlas en mayúsculas.

4. Encapsulamiento: mutators y accessors

El encapsulamiento es uno de los principios de la programación orientada a objetos y consiste en esconder el estado interno de un objeto detrás de métodos controlados. A los métodos que modifican un atributo (los `set`) se les llama *mutators*, y a los que lo leen (los `get`), *accessors*. La convención de nombres es directa: un mutator se llama `setNombreDelCampo`, recibe un parámetro del mismo tipo que el campo y no devuelve nada; un accessor se llama `getNombreDelCampo`, no recibe parámetros y devuelve el valor.

Cuando el parámetro del mutator se llama igual que el campo que modifica, hace falta la palabra clave `this` para distinguir uno de otro dentro del cuerpo del método:

Código 2: Uso de `this` cuando el parámetro repite el nombre del campo

```
1 public void setIdCarrera(int idCarrera) {  
2     this.idCarrera = idCarrera;  
3 }
```

Sin el `this` a la izquierda, la línea sería `idCarrera = idCarrera`, una asignación del parámetro a sí mismo que compila sin error mientras el campo de la clase nunca cambia. Es un error fácil de cometer y difícil de detectar precisamente porque no truena en tiempo de compilación.

Una clase que sigue este patrón por completo (campos privados, un constructor sin argumentos, y accessors y mutators públicos para cada campo) recibe el nombre de *JavaBean*. No es una curiosidad de nomenclatura: muchas herramientas de Java dan por hecho que las clases tienen esta forma, porque necesitan poder crear el objeto con el constructor vacío y llenarlo después llamando a los setters uno por uno. Esa es la razón por la que el constructor vacío importa tanto una vez que se llega a Hibernate o a JPA.

5. Constructores

Si una clase no declara ningún constructor, el compilador genera uno vacío de forma automática, que además invoca implícitamente a `super()` e inicializa los campos con sus valores por defecto (0 para los numéricos, `null` para los objetos). Ese constructor generado desaparece en cuanto se escribe cualquier constructor explícito, aunque tenga parámetros distintos; si se necesita seguir creando objetos sin argumentos, hay que declarar ese constructor vacío a mano.

Se puede evitar repetir la misma asignación de campos en varios constructores: uno puede llamar a otro con `this(...)` como primera línea de su cuerpo, delegando la parte común y añadiendo solo lo que le falta. Es el mismo principio que evita duplicar código en cualquier otra parte de la clase.

6. Herencia, clases abstractas e interfaces

Java permite que una clase extienda una sola superclase (**extends**). Esta restricción se explica con el llamado problema del diamante: si una clase pudiera heredar de dos clases que a su vez heredan de una tercera en común, no habría forma de decidir de cuál de las dos rutas hereda los miembros compartidos. La solución que ofrece el lenguaje son las interfaces, que sí se pueden implementar en cualquier número.

Una clase abstracta (**abstract class**) no se puede instanciar directamente, pero puede tener constructores, campos y métodos con implementación completa junto a métodos abstractos sin cuerpo, que las subclasses están obligadas a completar. Una interfaz, en cambio, no puede tener campos de instancia ni constructores; hasta Java 8 solo podía declarar firmas de método, y desde entonces también admite métodos **default** con implementación, pensados para no romper a las clases que ya implementaban la interfaz cuando se le agrega un método nuevo. La tabla 1 resume las diferencias entre ambas.

	Clase abstracta	Interfaz
Instanciación	No, aunque puede tener constructores	No, y no tiene constructores
Herencia	Una sola superclase	Una clase puede implementar varias
Miembros	Métodos abstractos y concretos, campos de cualquier tipo	Solo métodos (abstractos, default o static) y campos static final
Visibilidad por defecto	La que declare cada miembro	public y abstract de forma implícita

Tabla 1: Diferencias entre clase abstracta e interfaz

Esta distinción es la misma que separa a la interfaz de la implementación en el patrón DAO (Data Access Object): la interfaz declara qué operaciones existen (guardar, buscar, actualizar) sin decir cómo se hacen, y una clase como `CarreraDAO` provee la implementación concreta contra MySQL. Cambiar de manejador de base de datos, en teoría, solo debería requerir escribir otra implementación de la misma interfaz.

7. Enums

Un **enum** es un tipo de clase especial que solo se puede instanciar un número fijo de veces, una por cada constante que declara. Su constructor es implícitamente privado, porque nadie fuera del propio enum puede crear instancias nuevas, y cada instancia puede llevar sus propios campos y hasta sobrescribir un método declarado en el enum. El patrón encaja con cualquier campo que solo pueda tomar un conjunto fijo y conocido de valores, como el estatus de un registro o el rol de un usuario dentro de un sistema.

8. Excepciones: checked y unchecked

Todas las condiciones excepcionales en Java cuelgan de **Throwable**, que se divide en **Error** (fallas de la máquina virtual de las que no tiene sentido intentar recuperarse, como quedarse sin memoria) y **Exception** (situaciones que el programa puede y debe manejar), como se ve en la figura 1. Nunca se debe atrapar **Throwable** en general ni «tragarse» una excepción con un bloque **catch** vacío, porque eso oculta el problema en vez de resolverlo.

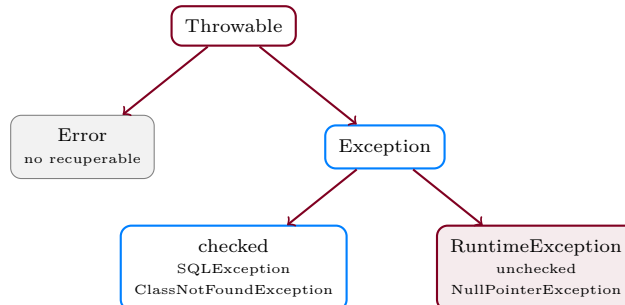


Figura 1: Jerarquía de **Throwable** y ubicación de las excepciones checked y unchecked

Dentro de **Exception** hay una segunda división que importa para cualquier código que dependa de recursos externos, como una conexión a base de datos. Las excepciones *unchecked* extienden **RuntimeException** (la más común es **NullPointerException**) y el compilador no obliga a tratarlas. Las excepciones *checked* son las que un método declara explícitamente con **throws**, y ahí el compilador sí exige que quien llame a ese método las atrape o las vuelva a declarar como propias. Es la razón por la que el bloque **try/catch** del método de conexión no es opcional:

Código 3: Manejo obligatorio de dos excepciones checked distintas

```
1 try {  
2     Class.forName(driverMySQL);  
3     conexion = DriverManager.getConnection(urlBD, usuario, clave);  
4 } catch (ClassNotFoundException | SQLException e) {  
5     // registrar en un log, nunca imprimir el stack trace completo  
6 }
```

ClassNotFoundException y **SQLException** son ambas checked, así que sin el **try/catch** el código no compila. La barra vertical entre los dos tipos de excepción se llama multi-catch y evita escribir dos bloques **catch** idénticos para tratarlas igual. Sobre qué hacer dentro del bloque, lo recomendable es no imprimir el *stack trace* completo en la consola de producción; lo correcto es mandarlo a un registro que alguien pueda revisar después.

9. Generics

Los generics se introdujeron en Java 5 para escribir clases que procesan cualquier tipo sin perder la verificación de tipos en tiempo de compilación. Un caso clásico es una clase **Pair<X, Y>** capaz de guardar dos valores de tipos distintos. Un caso más simple y frecuente es **List<Carrera>** como tipo de retorno de un método que trae todos los registros de una tabla: **List** es la clase genérica y **Carrera** es el tipo concreto con el que se está usando en ese punto del código.

10. Conclusión

Nada de esto es exótico: es la sintaxis con la que Java construye sus clases, con el vocabulario correcto para cada pieza. Distinguir un mutator de un accessor, saber por qué desaparece el constructor por defecto en cuanto se declara uno propio, entender qué hace que una clase sea un **JavaBean**, y separar una excepción checked de una unchecked son la base mínima para leer y escribir Java con criterio, antes de pasar a herramientas más complejas, como los frameworks de acceso a datos, que se construyen encima de estas reglas.

Angel Frausto Robles

11. Referencias Bibliográficas

- [1] Iuliana Cosmina. *Java for Absolute Beginners*. Apress, 2018. DOI: 10.1007/978-1-4842-3778-6.
- [2] Oracle Corporation. *The Java Tutorials: Exceptions*. URL: <https://docs.oracle.com/javase/tutorial/essential/exceptions/> (visitado 31-08-2026).